

GRADLE

20-Page Study Notes

Build Automation Tool • Java / Kotlin / Android / JVM

1. Introduction to Gradle

What is Gradle?

Gradle is an open-source build automation tool used to build, test, package, and publish software. It is especially popular in Java, Kotlin, Android, and JVM-based projects.

Why do we need a build tool?

A build tool automates repetitive tasks such as compiling source code, downloading dependencies, running tests, creating JAR/WAR files, and preparing applications for release.

Key idea

Instead of manually performing every build step, developers describe the project and its tasks in Gradle build files. Gradle then executes the required tasks in the correct order.

2. Gradle Architecture

Main components

Gradle works through a Gradle Wrapper, build scripts, projects, tasks, plugins, dependency configurations, and the Gradle build engine.

Typical flow

Developer → source code → build.gradle(.kts) → Gradle tasks → dependency resolution → compilation/testing → packaged artifact.

Gradle Daemon

The Gradle Daemon is a long-lived background process that keeps Gradle state warm between builds, improving build performance.

3. Installation and Gradle Wrapper

Installing Gradle

Gradle can be installed directly, but projects commonly use the Gradle Wrapper so every developer and CI server uses the project's intended Gradle version.

Wrapper commands

gradlew on Windows and ./gradlew on Linux/macOS execute Gradle through the project wrapper.

Useful commands

```
./gradlew tasks
```

```
./gradlew build
```

```
./gradlew test
```

```
./gradlew clean
```

4. Gradle Project Structure

Common structure

A Java project commonly contains `settings.gradle(.kts)`, `build.gradle(.kts)`, `gradle/wrapper`, `src/main`, and `src/test`.

Source folders

`src/main` contains application source code and resources. `src/test` contains automated tests.

settings file

`settings.gradle(.kts)` identifies the build and can define included subprojects in a multi-project build.

5. Build Scripts

Groovy vs Kotlin DSL

Gradle supports Groovy DSL files such as `build.gradle` and Kotlin DSL files such as `build.gradle.kts`.

Purpose

Build scripts declare plugins, repositories, dependencies, project metadata, and task configuration.

Example

```
plugins {  
    id 'java'  
}  
  
repositories { mavenCentral() }  
  
dependencies {  
    testImplementation 'org.junit.jupiter:junit-jupiter:5.10.0'  
}
```

6. Tasks in Gradle

What is a task?

A task represents a unit of work, such as compileJava, test, jar, clean, or build.

Task graph

Gradle creates a task graph from requested tasks and their dependencies. Only the tasks needed for the requested operation are executed.

Custom task

```
tasks.register('hello') {  
    doLast { println 'Hello Gradle' }  
}
```

7. Plugins

What are plugins?

Plugins extend Gradle by adding conventions, tasks, configurations, and capabilities.

Java plugin

The Java plugin adds standard tasks for compiling Java, processing resources, running tests, creating JARs, and checking the project.

Android plugin

Android projects use the Android Gradle Plugin, which adds Android-specific build, packaging, variant, and testing capabilities.

8. Dependencies

Dependency management

Gradle can download and manage external libraries instead of requiring developers to manually copy JAR files.

Repositories

Repositories are locations from which dependencies are resolved. Maven Central is a widely used repository.

Configurations

Common Java configurations include `implementation`, `compileOnly`, `runtimeOnly`, and `testImplementation`.

9. Dependency Resolution

Coordinates

A dependency is commonly identified by group, name/module, and version, for example `org.junit.jupiter:junit-jupiter:5.10.0`.

Transitive dependencies

A library may depend on other libraries. Gradle can resolve these transitive dependencies automatically.

Conflict resolution

When multiple dependency paths request different versions, Gradle applies dependency-resolution rules to select a version; projects can also explicitly control versions.

10. Build Lifecycle

Three phases

Gradle builds are generally understood through initialization, configuration, and execution.

Initialization

Gradle determines which projects participate in the build.

Configuration

Build scripts are evaluated and the task graph is prepared.

Execution

The selected tasks and their actions are executed.

11. Gradle vs Maven

Gradle

Uses flexible build logic, supports Groovy and Kotlin DSLs, and emphasizes incremental and configurable builds.

Maven

Uses XML-based POM files and a convention-driven lifecycle. Maven is widely used for Java dependency and build management.

Quick comparison

Gradle: build.gradle(.kts), DSL-based configuration, highly programmable. Maven: pom.xml, XML configuration, strong conventions.

Important

Both can use repositories such as Maven Central and both are widely used in Java/JVM ecosystems.

12. Incremental Builds

Meaning

Gradle avoids repeating work when inputs and outputs have not changed.

Why useful?

Incremental execution can reduce build time, especially in large projects.

Example

If one source file changes, Gradle can often compile only the affected work rather than recompiling everything.

13. Build Cache and Performance

Build cache

Gradle can reuse outputs from previous executions when task inputs match. This can speed up local and CI builds.

Performance techniques

Use the Gradle Wrapper, keep tasks incremental, avoid unnecessary work in configuration, use appropriate caching, and keep dependencies controlled.

Daemon

The Daemon also contributes to faster repeated builds by keeping a JVM process available.

14. Testing with Gradle

Testing

Gradle can integrate unit and integration tests into the build.

Typical command

```
./gradlew test
```

JUnit example

With the Java plugin and JUnit dependency configured, Gradle's test task can discover and run tests and produce test reports.

CI benefit

Automated tests can run on every commit or pull request through a CI system.

15. Packaging and Artifacts

Packaging

After compilation and testing, Gradle can package applications into artifacts such as JAR files.

Common tasks

jar creates a JAR for Java projects; build combines major verification and assembly steps; clean removes generated build outputs.

Artifact

An artifact is a build output that can be stored, shared, deployed, or published.

16. Multi-Project Builds

Meaning

A multi-project build contains multiple related Gradle subprojects in one build.

Use case

For example, a company application might have separate modules for API, core logic, database access, and web UI.

Benefits

Shared configuration, controlled project dependencies, and coordinated builds can simplify large codebases.

17. Gradle in CI/CD

CI/CD role

Gradle is commonly used inside continuous integration pipelines to compile code, execute tests, perform checks, and create artifacts.

Typical flow

Git push → CI server → ./gradlew build → tests → artifact → deployment.

Examples

Gradle can run in Jenkins, GitHub Actions, GitLab CI, and other CI environments. The Gradle Wrapper helps keep the build reproducible.

18. Useful Commands & Troubleshooting

Essential commands

```
./gradlew tasks
./gradlew clean
./gradlew build
./gradlew test
./gradlew dependencies
```

Troubleshooting

Read the first meaningful error, run with more diagnostic output when necessary, verify dependency versions, check repositories, and confirm the JDK version.

Useful flags

--stacktrace provides stack traces; --info increases logging; --scan can provide detailed build insights when enabled.

19. Advantages, Limitations & Best Practices

Advantages

Fast incremental builds, flexible DSLs, strong dependency management, plugin ecosystem, multi-project support, and good CI integration.

Limitations

Flexible configuration can become complex; large builds require disciplined conventions and dependency management.

Best practices

Use the Wrapper, pin important versions, keep build logic modular, avoid unnecessary custom tasks, manage dependencies carefully, and run tests automatically.

20. Exam Revision & Viva Questions

One-line definition

Gradle is a build automation tool that automates compilation, testing, dependency management, packaging, and publishing.

Important questions

1. What is Gradle?
2. Why is Gradle used?
3. What is Gradle Wrapper?
4. What is a Gradle task?
5. What is a plugin?
6. Difference between Gradle and Maven?
7. What is dependency management?
8. What is incremental build?
9. What is the Gradle Daemon?
10. How is Gradle used in CI/CD?

Quick answers

Wrapper: ensures a project uses its intended Gradle version. Task: unit of build work. Plugin: adds build functionality. Dependency: external library required by a project. Incremental build: executes only necessary changed work.